UE（Unreal Engine）Slate 学習ノート① — カスタムエディタを開く①

```
/**
 * Creates an instance of a Kismet editor object.  Only virtual so that it can be called across the DLL boundary.
 *
 * Note: This function should not be called directly, use one of the following instead:
 *  - FKismetEditorUtilities::BringKismetToFocusAttentionOnObject
 *  - GEditor->GetEditorSubsystem<UAssetEditorSubsystem>()->OpenEditorForAsset
 *
 * @param   Mode                    Mode that this editor should operate in
 * @param   InitToolkitHost         When Mode is WorldCentric, this is the level editor instance to spawn this editor within
 * @param   Blueprint               The blueprint object to start editing
 * @param   bShouldOpenInDefaultsMode   If true, the editor will open in defaults editing mode
 *
 * @return  Interface to the new Blueprint editor
 */
virtual TSharedRef<IBlueprintEditor> CreateBlueprintEditor(const EToolkitMode::Type Mode, const TSharedPtr< IToolkitHost >& InitToolkitHost, UBlueprint* Blueprint, bool bShouldOpenInDefaultsMode = false);
virtual TSharedRef<IBlueprintEditor> CreateBlueprintEditor( const EToolkitMode::Type Mode, const TSharedPtr< IToolkitHost >& InitToolkitHost, const TArray< UBlueprint* >& BlueprintsToEdit );
```

知乎 @hareonnna

BlueprintEditorModule を確認していると、コメント内に Blueprint エディタを開く方法が 2 通りあることが分かりました。
ここでは、
GEditor->GetEditorSubsystem<UAssetEditorSubsystem>()->OpenEditorForAsset
を使用して呼び出す方法を選択しています。
OpenEditorForAsset 関数の内部では、コアとなる処理は **IAssetTypeActions** を通じて **OpenAssetEditor** 関数が呼び出される仕組みになっていると考えられます。

```
FAssetToolsModule& AssetToolsModule = FModuleManager::LoadModuleChecked<FAssetToolsModule>(TEXT("AssetTools"));
```

```
TWeakPtr<IAssetTypeActions> AssetTypeActions = AssetToolsModule.Get().GetAssetTypeActionsForClass(Asset->GetClass());
```

```
if (AssetTypeActions.IsValid())
{
    TArray<UObject*> AssetsToEdit;
    AssetsToEdit.Add(Asset);

    // Some assets (like UWorlds) may be destroyed and recreated as part of opening. To protect against this, keep the path to the asset and try to re-find it if it disappeared.
    TWeakObjectPtr<UObject> WeakAsset = Asset;
    const FString AssetPath = Asset->GetPathName();

    AssetTypeActions.Pin()->OpenAssetEditor(AssetsToEdit, ActualToolkitMode == EToolkitMode::WorldCentric ? OpenedFromLevelEditor : TSharedPtr<IToolkitHost>());

    // If the Asset was destroyed, attempt to find it if it was recreated
    if (!WeakAsset.IsValid() && !AssetPath.IsEmpty())
    {
        Asset = FindObject<UObject>(nullptr, *AssetPath);
    }

    AssetEditorOpenedEvent.Broadcast(Asset);
}
```

知乎 @hareonnna

GetAssetTypeActionsForClass 関数では、**AssetTypeActionsList** 配列を走査することで **IAssetTypeActions** を取得します。
Blueprint には既に Blueprint 用のデフォルトの **IAssetTypeActions** が存在しているため、カスタムエディタを開くには、新しい **IAssetTypeActions** を作成し、その **Supported クラス** を **UBlueprint** に変更する必要があります。

```
class FFNewAssetAction :public FAssetTypeActions_Base
{
public:
    virtual uint32 GetCategories() override;
    virtual FText GetName()const override;
    virtual UClass*GetSupportedClass() const override;
    virtual FColor GetTypeColor() const override;
    virtual void OpenAssetEditor(const TArray<UObject*>& InObjects, TSharedPtr<IToolkitHost> EditWithinLevelEditor = TSharedPtr<IToolkitHost>()) override;
};
```

知乎 @hareonnna

カスタム IAssetTypeActions のヘッダーファイル

```
#include "Framework/Docking/TabManager.h"
#include "Widgets\Layout\SConstraintCanvas.h"
uint32 FFNewAssetAction::GetCategories()
{
    return EAssetTypeCategories::Basic;
}

FText FFNewAssetAction::GetName() const
{
    return FText::FromString(GetSupportedClass()->GetName());
}

UClass* FFNewAssetAction::GetSupportedClass() const
{
    return UBlueprint::StaticClass();
}

FColor FFNewAssetAction::GetTypeColor() const
{
    return FColor::Red;
}
```

知乎 @hareonnna

作成した後は、自分で定義した **IAssetTypeActions** を **FAssetToolsModule** モジュールの **AssetTypeActionsList** 配列に登録する必要があります。これにより取得できるようになります。

```
IAssetTools&AssetTools=FModuleManager::LoadModuleChecked<FAssetToolsModule>("AssetTools").Get();
Action=MakeShareable(new FFNewAssetAction);
AssetTools.RegisterAssetTypeActions(Action.ToSharedRef());
```

その後、**OpenEditorForAsset()** を呼び出します

```
UObject*blue=LoadObject<UObject>(nullptr, TEXT("/Game/NewBlueprint.NewBlueprint"));
GEditor->GetEditorSubsystem<UAssetEditorSubsystem>()->OpenEditorForAsset(blue,EToolkitMode::Standalone,{},true);
```

最後に重要なのは、**カスタムエディタの生成**です。ここでは **FAssetEditorToolkit** を継承したクラスを独自に定義する必要があります。
そのクラス内の **InitAssetEditor** を使用して、カスタムエディタを生成します。元々、Blueprint の **IAssetTypeActions** でも同様の方法が用いられています。
InitAssetEditor の中では、エディタ全体の基本的な構成（フレームワーク）がどのように生成されるか、さらにメニューバーやツールバーがどのように作成されるか、各サブ項目がどのように生成され適切な位置へ挿入されるかが分かります。また、このカスタムエディタに対して独自の拡張（カスタマイズ）を行うことも可能です。

UE Slate 学習ノート 2 - カスタムエディタを開く②

前回は **InitAssetEditor()** について触れましたが、今回はこの関数を重点的に調べていきます。[1]
まず、**FAssetEditorToolkit** を継承したクラスを独自に作成します。

```
*/
class FCustomNormalDistributionEditorToolkit : public FAssetEditorToolkit
{
public:
    // 外部调用的入口，它可以是任意名字，可以具有任意参数。
    void InitEditor(const TArray<UObject*>& InObjects);
    // 必须实现的虚函数
    virtual void RegisterTabSpawners(const TSharedRef<class FTabManager>& TabManager) override;
    virtual void UnregisterTabSpawners(const TSharedRef<class FTabManager>& TabManager) override;
    virtual FName GetToolkitFName() const override { return "CustomNormalDistributionEditor"; }
    virtual FText GetBaseToolkitName() const override { return INVTEXT("Custom Normal Distribution Editor"); }
    virtual FString GetWorldCentricTabPrefix() const override { return "Custom Normal Distribution"; }
    virtual FLinearColor GetWorldCentricTabColorScale() const override { return {}; }
```

知乎 @hareonnna

InitEditor 関数の中では **FAssetEditorToolkit::InitAssetEditor()** を呼び出します。
また、カスタムクラスの **InitEditor** 関数は、前回触れた **IAssetTypeActions** の **OpenAssetEditor()** から呼び出されます。
ここでは、関心のあるコードのみに注目します。ウィンドウをどのように追加し、最前面に表示するかについては、**Tab** の章で説明します。
ソースコードを確認すると、エディタ全体の表示内容は、実際には **SDockTab** によって表示される内容で構成されていることが分かります。

```
// Create a new SlateToolkitHost
NewMajorTab = SNew(SDockTab)
    .ContentPadding(0.0f)
    .TabRole(ETabRole::MajorTab)
    .ToolTip(IDocumentation::Get()->CreateToolTip(ToolTipText, nullptr, DocLink, GetToolkitFName().ToString()))
    .Icon(this, &FAssetEditorToolkit::GetDefaultTabIcon)
    .TabColorScale(this, &FAssetEditorToolkit::GetDefaultTabColor)
    .Label(Label)
    .LabelSuffix(LabelSuffix);
```

知乎 @hareonnna

SDockTab を初期化する

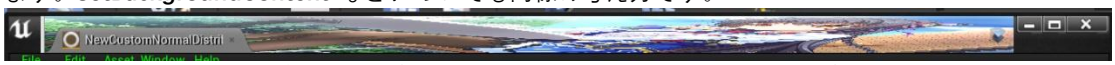
```
IIntroTutorials& IntroTutorials = FModuleManager::LoadModuleChecked<IIntroTutorials>(TEXT("IntroTutorials"));
TSharedRef<SWidget> TutorialWidget = IntroTutorials.CreateTutorialsWidget(GetToolkitContextFName(), NewMajorTab->GetParentWindow());

NewMajorTab->SetRightContent(
        SNew(SHorizontalBox)
        +SHorizontalBox::Slot()
        .AutoWidth()
        .Padding(8.0f, 0.0f, 8.0f, 0.0f)
        .VAlign(VAlign_Center)
        [
            TutorialWidget
        ]
    );
```

知乎 @hareonnna

チュートリアル用の SWidget を生成する

つまり、上図にあるものがそれに該当します。そのため、**SetRightContent** の意味も理解できると思います。**SetBackgroundContent** などについても同様の考え方です。



その後ろにあるぼかし（ブラー）の背景が **BackgroundContent** に該当します。
そして次に重要なのが、**SDockTab** によって主に表示されるコンテンツ部分です。

```
const TSharedRef<FTabManager> NewTabManager = FGlobalTabmanager::Get()->NewTabManager( NewMajorTab.ToSharedRef() );
NewTabManager->SetOnPersistLayout(FTabManager::FOnPersistLayout::CreateRaw(this, &FAssetEditorToolkit::HandleTabManagerPersistLayout));
this->TabManager = NewTabManager;

NewMajorTab->SetContent
(
    SAssignNew( NewStandaloneHost, SStandaloneAssetEditorToolkitHost, NewTabManager, AppIdentifier )
    .OnRequestClose(this, &FAssetEditorToolkit::OnRequestClose)
);
```

知乎 @hareonnna

SStandaloneAssetEditorToolkitHost クラスは、ひとまず **SWidget** の一種として考えることができます。上記と同様に、表示する内容を異なる箇所に設定しているだけです。
したがって、**SStandaloneAssetEditorToolkitHost** クラスにコンテンツを設定することで、実際に表示される主要な内容が決まります。

```
ToolkitCommands->MapAction(
    FAssetEditorCommonCommands::Get().SaveAsset,
    FExecuteAction::CreateSP( this, &FAssetEditorToolkit::SaveAsset_Execute ),
    FCanExecuteAction::CreateSP( this, &FAssetEditorToolkit::CanSaveAsset ));

ToolkitCommands->MapAction(
    FAssetEditorCommonCommands::Get().SaveAssetAs,
    FExecuteAction::CreateSP( this, &FAssetEditorToolkit::SaveAssetAs_Execute ),
    FCanExecuteAction::CreateSP( this, &FAssetEditorToolkit::CanSaveAssetAs ));

ToolkitCommands->MapAction(
    FGlobalEditorCommonCommands::Get().FindInContentBrowser,
    FExecuteAction::CreateSP( this, &FAssetEditorToolkit::FindInContentBrowser_Execute ),
    FCanExecuteAction::CreateSP( this, &FAssetEditorToolkit::CanFindInContentBrowser ));
```

知乎 @hareonnna

さらに下へ進むと、多くの **FUICommandList** が定義されており、**FUICommandInfo** と **FUIAction** のバインドが行われていることが分かります。これらはメニューバーやツールバーを初期化する際に必要となります（ただし、多くが実際には使用されていないようにも見えます）。

1. メニューバーの構築方法

まず **SetupInitialContent** 関数に入ります。ここでは、先にコアとなる関数 **GenerateWidget()** を確認します。前段階で行われている処理についても、**GenerateWidget** 関数内で触れられています。

```
// Create menus
if (ToolkitMode == EToolkitMode::Standalone)
{
    AddMenuExtender(GetSharedMenuExtensibilityManager()->GetAllExtenders(ToolkitCommands, EditingObjects));

    TSharedRef<FTabManager::FLayout> LayoutToUse = FLayoutSaveRestore::LoadFromConfig(GEditorLayoutIni, StandaloneDefaultLayout);

    // Actually create the widget content
    NewStandaloneHost->SetupInitialContent(LayoutToUse, NewMajorTab, bCreateDefaultStandaloneMenu);
}
```

知乎 @hareonnna

```
TSharedRef<SWidget> FMainMenu::MakeMainMenu(const TSharedPtr<FTabManager>& TabManager, const FName MenuName, FToolMenuContext& ToolMenuContext)
{
    // Cache all project names once
    FMainFrameActionCallbacks::CacheProjectNames();

    FMainMenu::RegisterMainMenu();

    ToolMenuContext.AppendCommandList(FMainFrameCommands::ActionList);

    USlateTabManagerContext* ContextObject = NewObject<USlateTabManagerContext>();
    ContextObject->TabManager = TabManager;
    ToolMenuContext.AddObject(ContextObject);

    // Create the menu bar!
    TSharedRef<SWidget> MenuBarWidget = UToolMenus::Get()->GenerateWidget(MenuName, ToolMenuContext);
    if (MenuBarWidget != SNullWidget::NullWidget)
    {
        // Tell tab-manager about the multi-box for platforms with a global menu bar
        TSharedRef<SMultiBoxWidget> MultiBoxWidget = StaticCastSharedRef<SMultiBoxWidget>(MenuBarWidget);
```

知乎 @hareonnna

GenerateMenu() の中では、
return GenerateMenuFromHierarchy(CollectHierarchy(Name), InMenuContext);
が呼び出されています。
まず **CollectHierarchy()** から見ていきます。この関数の主な役割は、自身が登録した **UToolMenu** と、その親クラスにあたる **UToolMenu** をすべて配列に追加し、その配列を返すことです。

```
        }
    }
}

UToolMenu* Current = FindMenu(AdjustedMenuName);
if (!Current)  ≤ 2ms elapsed
    AdjustedMenuName    AssetEditor.CustomNormalDistributionEditor.MainMenu
{
    UE_LOG(LogToolMenus, Warning, TEXT("Failed to find menu: %s for %s"), *AdjustedMenuName.ToString(), *InName.ToString());
    return TArray<UToolMenu*>();
}

if (Result.Contains(Current))
{
    UE_LOG(LogToolMenus, Warning, TEXT("Infinite loop detected in tool menu: %s"), *InName.ToString());
    return TArray<UToolMenu*>();
}

Result.Add(Current);
```

知乎 @hareonnna

FindMenu は、登録されているメニューを検索するための関数です。
まず、自身で登録した **UToolMenu** を検索しますが、これは **SetupInitialContent** 関数の冒頭で既に登録されています。

```
const FName AssetEditorMenuName = GetMenuName();
if (!UToolMenus::Get()->IsMenuRegistered(AssetEditorMenuName))
{
    UToolMenu* Menu = UToolMenus::Get()->RegisterMenu(AssetEditorMenuName, "MainFrame.MainMenu");
                                                      AssetEditorMenuName    AssetEditor.CustomNormalDistributionEditor.MainMenu
    if (bCreateDefaultStandaloneMenu)
    {
        CreateDefaultStandaloneMenuBar(Menu);
    }
}

DefaultMenuWidget = SNullWidget::NullWidget;
```

知乎 @hareonnna

その後、親クラスの名前を **CurrentMenuName** に渡し、さらに **AdjustedMenuName** に渡して親クラスの **UToolMenu** を検索します。そして、親クラス名が空になるまでこの処理を繰り返し、ループを終了します。
ここでの親クラス名は **MainFrame.MainMenu** であり、これは先ほど登録した際に、親クラスが **MainFrame** として既に設定されています。

```
            SubstitutedMenuNames.Add(CurrentMenuName);
        }
    }
}

UToolMenu* Current = FindMenu(AdjustedMenuName);  ≤ 1ms elapsed
    AdjustedMenuName    MainFrame.MainMenu
if (!Current)
{
    UE_LOG(LogToolMenus, Warning, TEXT("Failed to find menu: %s for %s"), *AdjustedMenuName.ToString(), *InName.ToString());
    return TArray<UToolMenu*>();
}

if (Result.Contains(Current))
{
    UE_LOG(LogToolMenus, Warning, TEXT("Infinite loop detected in tool menu: %s"), *InName.ToString
    return TArray<UToolMenu*>();
```

知乎 @hareonnna

親クラスに関するすべての登録は、**MakeMainMenu** の中で行われています。

```
TSharedRef<SWidget> FMainMenu::MakeMainMenu(const TSharedPtr<FTabManager>& TabManager, const FName MenuName, FToolMenuContext& ToolMenuContext)
{
    // Cache all project names once
    FMainFrameActionCallbacks::CacheProjectNames();

    FMainMenu::RegisterMainMenu();
                FMainMenu::RegisterMainMenu    0x00007ffb50e6e8f0 {UE4Editor-MainFrame.dll!FMainMenu::RegisterMainMenu(void)}
    ToolMenuContext.AppendCommandList(FMainFrameCommands::ActionList);

    USlateTabManagerContext* ContextObject = NewObject<USlateTabManagerContext>();
    ContextObject->TabManager = TabManager;
    ToolMenuContext.AddObject(ContextObject);

    // Create the menu bar!
    TSharedRef<SWidget> MenuBarWidget = UToolMenus::Get()->GenerateWidget(MenuName, ToolMenuContext);
    if (MenuBarWidget != SNullWidget::NullWidget)
    {
        // Tell tab-manager about the multi-box for platforms with a global menu bar
        TSharedRef<SMultiBoxWidget> MultiBoxWidget = StaticCastSharedRef<SMultiBoxWidget>(MenuBarWidget);
```

知乎 @hareonnna

RegisterMainMenu を登録する

次に行うのは、**AssembleMenuHierarchy** の中で、親クラスと自分で登録した **UToolMenu** を組み合わせる処理です。

```
[30]  {MainFrame.MainTabMenu.File,0x000001d924eeb9c0 (Name=ToolMenu_34)}
[31]  {MainFrame.MainMenu.Edit,0x000001d924eeb880 (Name=ToolMenu_35)}
[32]  {MainFrame.MainMenu.Window,0x000001d924eeb740 (Name=ToolMenu_36)}
[33]  {MainFrame.MainMenu.Help,0x000001d924eeb600 (Name=ToolMenu_37)}
[34]  {MainFrame.MainMenu,0x000001d924eeb4c0 (Name=ToolMenu_38)}
[35]  {AssetEditor.CustomNormalDistributionEditor.MainMenu,0x000001d90e7056c0 (Name=ToolMenu_41)}
[36]  {AssetEditor.CustomNormalDistributionEditor.MainMenu.File,0x000001d90e6e8400 (Name=ToolMenu_42)}
[37]  {AssetEditor.CustomNormalDistributionEditor.MainMenu.Help,0x000001d90e6e9580 (Name=ToolMenu_43)}
```

知乎 @hareonnna

確認すると、親クラスには 4 つの **Section**（**File**, **Edit**, **Window**, **Help**）があります。
一方、自分側では **File** と **Help** を持っています（実際には **Asset** もありますが、こちらは登録されていません）。
SetupInitialContent 関数内の **CreateDefaultStandaloneMenuBar** 関数において、これらの項目が追加されています。自分側の **File** と **Help** の登録も、この関数内で行われています。

```
Local::ExtendFileMenu(MenuBar);

// Add the "Asset" menu, if we're editing an asset
MenuBar->FindOrAddSection(NAME_None).AddDynamicEntry("DynamicAssetEntry", FNewToolMenuSectionDelegate::CreateLambda([](FToolMenuSection& InSection)
{
    UAssetEditorToolkitMenuContext* Context = InSection.FindContext<UAssetEditorToolkitMenuContext>();
    if (Context->Toolkit.Pin()->IsActuallyAnAsset())
    {
        InSection.AddSubMenu(
            "Asset",
            LOCTEXT("AssetMenuLabel", "Asset"),     // @todo toolkit major: Either use "Asset", "File", or the asset type name e.g. "Blueprint" (Also update custom pull-down
            LOCTEXT("AssetMenuLabel_ToolTip", "Opens a menu with commands for managing this asset"),
            FNewToolMenuDelegate::CreateStatic(&Local::FillAssetMenu)
            ).InsertPosition = FToolMenuInsert("Edit", EToolMenuInsertType::After);
    }
}));
```

知乎 @hareonnna

その後、**GenerateMenuFromHierarchy** 関数内の **AssembleMenuHierarchy** 関数において、2 つの **UToolMenu** の **Section** を統合する処理が行われます。

```
UToolMenu* UToolMenus::GenerateMenuFromHierarchy(const TArray<UToolMenu*>& Hierarchy, const FToolMenuContext& InMenuContext)
{
    UToolMenu* GeneratedMenu = NewObject<UToolMenu>(this);

    if (Hierarchy.Num() > 0)
    {
        GeneratedMenu->InitGeneratedCopy(Hierarchy[0], Hierarchy.Last()->MenuName, &InMenuContext);
        for (UToolMenu* HiearchyItem : Hierarchy)
        {
            if (HiearchyItem && !HiearchyItem->bExtendersEnabled)
            {
                GeneratedMenu->SetExtendersEnabled(false);
                break;
            }
        }
        AssembleMenuHierarchy(GeneratedMenu, Hierarchy);
    }
```

知乎 @hareonnna

その後、**GenerateWidget** 関数に進みます。引数である *UToolMenu GeneratedMenu** がどのように生成されるかが分かると、ここで特に説明することはあまりありません。メニューバーを生成する処理であるため、このステップへと進みます。

```
}
else if (GeneratedMenu->MenuType == EMultiBoxType::MenuBar)
{
    FMenuBarBuilder MenuBarBuilder(GeneratedMenu->Context.CommandList, GeneratedMenu->Context.GetAllExtenders(), GeneratedMenu->StyleSet, GeneratedMenu->MenuNa
    MenuBarBuilder.SetExtendersEnabled(GeneratedMenu->bExtendersEnabled);
    PopulateMenuBarBuilder(MenuBarBuilder, GeneratedMenu);
    TSharedRef<SWidget> Result = MenuBarBuilder.MakeWidget();
    GeneratedMenuWidget.Widget = Result;
    return Result;
}
```

PopulateMenuBarBuilder 関数の中では、先ほど統合した **Section** に含まれる **Block** が一つずつ **FMenuBarBuilder** に挿入されます。
また、拡張機能（Extender）が追加されている場合は、**ApplyHook()** 関数の中で、その拡張機能にバインドされたデリゲートが呼び出されます。

2. ツールバーの構築方法

メニューバーの場合と同様です。

```
// Create toolbars
AddToolbarExtender(GetSharedToolBarExtensibilityManager()->GetAllExtenders(ToolkitCommands, EditingObjects));

if (bCreateDefaultToolbar)
{
    GenerateToolbar();
}
```

特に説明するほどのコードはないため、ここでは省略します。
次の節では、エディタをビューポートに追加する方法について説明します。
Reference:
《调教 UE5：编辑器拓展指南》Tab 探索 https://zhuanlan.zhihu.com/p/628655599

UE Slate 学習ノート 3 - カスタムエディタを開く③

FAssetEditorToolkit::InitAssetEditor 関数では、**FGlobalTabmanager::Get()->InsertNewDocumentTab** を通じて、カスタムエディタをビューポートに表示します。
この関数を理解する前に、まず最もシンプルにタブを呼び出してビューポートに表示する方法である **InvokeTab()** を確認します。
InvokeTab の内部では **InvokeTab_Internal** 関数が呼び出されており、コメントにはこの関数の役割が説明されています。

```
// Tab Spawning Rules:
//
//     * Find live instance --yes--> use it.
//         |no
//         v
//     * [non-Document only]
//       Find closed instance with matching TabId --yes--> restore it.
//         |no
//         v
//     * Find any tab of matching TabType (closed or open) --yes--> spawn next to it.
//         | no
//         v
//     * Is a nomad tab and we are NOT the global tab manager --yes--> try to invoke in the global tab manager
//         | no
//         v
//     * Spawn in a new window.
```

知乎 @hareonnna

InvokeTab の処理フロー

1. インスタンスが存在するかを確認

まず、**RegisterTabSpawner** によって登録が行われているかを確認します。すでに登録されている場合は、その Tab が既に呼び出されているかどうかをチェックします。

一度呼び出された Tab は、その **TabManager** 内の **FTabSpawnerEntry** において、開かれた **SpawnedTabPtr** として保存されています。

```
TSharedPtr<FTabSpawnerEntry> Spawner = FindTabSpawnerFor(TabId.TabType);

if ( !Spawner.IsValid() )
{
    UE_LOG(LogTabManager, Warning, TEXT("Cannot spawn tab because no spawner is registered for '%s'"), *(TabId.ToString()));
}
else
{
    TSharedPtr<SDockTab> ExistingTab = Spawner->OnFindTabToReuse.IsBound()
        ? Spawner->OnFindTabToReuse.Execute( TabId )
        : Spawner->SpawnedTabPtr.Pin();

    if ( ExistingTab.IsValid() )
    {
        if ( !ExistingTab->IsActive() )
        {
            // Draw attention to this tab if it didn't already have focus
            DrawAttention( ExistingTab.ToSharedRef() );
        }
        return ExistingTab.ToSharedRef();
    }
}
```

知乎 @hareonnna

後続の **RestoreArea_Helper()** の中で **SpawnTab()** が呼び出され、**SpawnedTabPtr** に値が設定されていることが分かります。ここで代入される値は、**RegisterTabSpawner** によって登録された際のデリゲート **FOnSpawnTab** の戻り値に相当します。

```
TSharedPtr<SDockTab> FTabManager::SpawnTab(const FTabId& TabId, const TSharedPtr<SWindow>& ParentWindow, const bool bCanOutputBeNullptr)
{
    TSharedPtr<SDockTab> NewTabWidget;

    // Whether or not the spawner overrode the ability for the tab to even spawn.  This is not a failure case.
    bool bSpawningAllowedBySpawner = true;
    // Do we know how to spawn such a tab?
    TSharedPtr<FTabSpawnerEntry> Spawner = FindTabSpawnerFor(TabId.TabType);
    if ( Spawner.IsValid() )
    {
        if (Spawner->CanSpawnTab.IsBound())
        {
            bSpawningAllowedBySpawner = Spawner->CanSpawnTab.Execute(FSpawnTabArgs(ParentWindow, TabId));
        }

        if (bSpawningAllowedBySpawner && (!Spawner->SpawnedTabPtr.IsValid() || Spawner->OnFindTabToReuse.IsBound()))
        {
            NewTabWidget = Spawner->OnSpawnTab.Execute(FSpawnTabArgs(ParentWindow, TabId));
            NewTabWidget->SetLayoutIdentifier(TabId);
            NewTabWidget->ProvideDefaultLabel(Spawner->GetDisplayName().IsEmpty() ? FText::FromName(Spawner->TabType) : Spawner->GetDisplayName());
            NewTabWidget->ProvideDefaultIcon(Spawner->GetIcon().GetIcon());

            // The spawner tracks that last tab it spawned
            Spawner->SpawnedTabPtr = NewTabWidget;
```

知乎 @hareonnna

2. 閉じられた CollapsedDockAreas (FArea) の中から、該当する TabId を持つ Stack（SDockingTabStack） を探す

つまり、この Tab が過去に一度開かれ、その後閉じられた場合、**CollapsedDockAreas** に閉じられた **DockArea** の情報が記録されます。

FindPotentiallyClosedTab の中の **FindTabInCollapsedAreas** を確認すると、ここで渡される **TabMatcher** は述語（predicate）として使用されます。このクラスは **() 演算子** をオーバーロードしています。

FindTabInCollapsedAreas 関数では、まず **Area** を走査し、その後 **Area / Splitter** をたどりながら、それぞれが持つ **Stack** を走査します。**Stack** の中には **Tabs 配列** が保持されています。

```
TSharedPtr<FTabManager::FStack> FTabManager::FindTabUnderNode( const FTabMatcher& Matcher, const TSharedRef<FTabManager::FLayoutNode>& NodeToSearchUnder )
{
    TSharedPtr<FTabManager::FStack> NodeAsStack = NodeToSearchUnder->AsStack();
    TSharedPtr<FTabManager::FSplitter> NodeAsSplitter = NodeToSearchUnder->AsSplitter();

    if (NodeAsStack.IsValid())
    {
        const int32 TabIndex = NodeAsStack->Tabs.IndexOfByPredicate(Matcher);
        if (TabIndex != INDEX_NONE)
        {
            return NodeAsStack;
        }
        else
        {
            return TSharedPtr<FTabManager::FStack>();
        }
    }
    else
    {
        ensure( NodeAsSplitter.IsValid() );
        TSharedPtr<FTabManager::FStack> StackWithTab;
        for ( int32 ChildIndex=0; !StackWithTab.IsValid() && ChildIndex < NodeAsSplitter->ChildNodes.Num(); ++ChildIndex)
        {
            StackWithTab = FindTabUnderNode( Matcher, NodeAsSplitter->ChildNodes[ChildIndex] );
        }
```

知乎 @hareonnna

TArray 配列では **IndexOfByPredicate** を呼び出して、この Tab が配列に含まれているかどうかを判定します。
戻り値が INDEX_NONE の場合、その Stack 内には該当する Tab が存在しないことを意味します。
また、**IndexOfByPredicate** に渡されるのは述語（**predicate**）であり、条件に一致する要素を検索するために使用されます。

```
*/
template <typename Predicate>
SizeType IndexOfByPredicate(Predicate Pred) const
{
    const ElementType* RESTRICT Start = GetData();
    for (const ElementType* RESTRICT Data = Start, *RESTRICT DataEnd = Start + ArrayNum; Data != DataEnd; ++Data)
    {
        if (Pred(*Data))
        {
            return static_cast<SizeType>(Data - Start);
        }
    }
    return INDEX_NONE;
}
```

知乎 @hareonnna

SizeType は InAllocator::SizeType を指します。

```
/// @cond DOXYGEN_WARNINGS
template<int IndexSize> class TSizedDefaultAllocator;
using FDefaultAllocator = TSizedDefaultAllocator<32>;
using FDefaultAllocator64 = TSizedDefaultAllocator<64>;
class FDefaultSetAllocator;

class FString;

template<typename T, typename Allocator = FDefaultAllocator> class TArray;
template<typename T> using TArray64 = TArray<T, FDefaultAllocator64>;
template<typename T> class TArrayView;
template<typename T> using TConstArrayView = TArrayView<const T>;
template<typename T> class TTransArray;
template<typename KeyType, typename ValueType, bool bInAllowDuplicateKeys> struct TDefaultMapHashableKeyFuncs;
template<typename KeyType, typename ValueType, typename SetAllocator = FDefaultSetAllocator, typename KeyFuncs = TDefaultMapHashableKeyFuncs<KeyType, ValueType, false> > class TMap;
template<typename KeyType, typename ValueType, typename SetAllocator = FDefaultSetAllocator, typename KeyFuncs = TDefaultMapHashableKeyFuncs<KeyType, ValueType, true > > class TMultiMap;
template <typename T = void > struct TLess;
template <typename> struct TTypeTraits;
template<typename KeyType, typename ValueType, typename ArrayAllocator = FDefaultAllocator, typename SortPredicate = TLess<typename TTypeTraits<KeyType>::ConstPointerType> > class TSortedMap;
template<typename ElementType, bool bInAllowDuplicateKeys = false> struct DefaultKeyFuncs;
template<typename InElementType, typename KeyFuncs = DefaultKeyFuncs<InElementType>, typename Allocator = FDefaultSetAllocator> class TSet;
/// @endcond
```

知乎 @hareonnna

No issues found

```
struct TBitsToSizeType
{
    static_assert(IndexSize, "Unsupported allocator index size.");
};

template <> struct TBitsToSizeType<8>  { using Type = int8; };
template <> struct TBitsToSizeType<16> { using Type = int16; };
template <> struct TBitsToSizeType<32> { using Type = int32; };
template <> struct TBitsToSizeType<64> { using Type = int64; };

/** The indirect allocation policy always allocates the elements indirectly. */
template <int IndexSize>   <T> Provide sample template arguments for IntelliSense
class TSizedHeapAllocator
{
public:
    using SizeType = typename TBitsToSizeType<IndexSize>::Type;

    enum { NeedsElementType = false };
    enum { RequireRangeCheck = true };

    class ForAnyElementType
```

知乎 @hareonnna

したがって、**SizeType** は int32 となり、戻り値は **TArray** 配列内で何番目の要素であるかを示します。
ElementType は **FTab** を表し、**Pred()** は **FTabMatcher** によってオーバーロードされた () 演算子 を呼び出します。一致する要素が見つかった場合、アドレスの差分からオフセットが求められ、**TArray** 内の位置（インデックス）が分かります。
もし閉じられた **Area** 内で該当する **Tab** が見つからない場合、その後はアクティブな **Area** の中から検索します。それでも対応する **Tab** が見つからない場合、次のステップへ進みます。

3.新しい SWindow の中で開始する

```
{
    // No layout info about this tab found: start
    TSharedRef<FArea> NewAreaForTab = FTabManager::NewArea(FTabManager::GetDefaultTabWindowSize(TabId))
    ->Split
    (
        FTabManager::NewStack()
        ->AddTab( TabId, ETabState::OpenedTab )
    );

    TSharedPtr<SDockingArea> DockingArea = RestoreArea(NewAreaForTab, GetPrivateApi().GetParentWindow());
    if (DockingArea && DockingArea->GetAllChildTabs().Num() > 0)
    {
        const TSharedPtr<SDockTab> NewlyOpenedTab = DockingArea->GetAllChildTabs()[0];
        check(NewlyOpenedTab.IsValid());
        return NewlyOpenedTab.ToSharedRef();
    }
    else
```

知乎 @hareonnna

コアとなる処理のほとんどは **RestoreArea()** の中にあります。基本的には走査と再帰処理が行われます。
引数として渡される **FLayoutNode** から、まず Area を取得します。
bDockNeedsNewWindow が true の場合、SNew によって新しいウィンドウを生成し、**FSlateApplication::Get().AddWindow** を呼び出してそのウィンドウを追加します。
さらに、**FSlateApplication::Get().AddWindow(NewWindow)->SetContent** によって **SDockingArea** の内容を設定します。
SArea は **SDockingArea** に値を設定するために用いられ、**RestoreSplitterContent** を通じて **SDockingArea** の内容が構築されます。

```
TArray<TSharedRef<SDockingNode>> DockingNodes;
if (CanRestoreSplitterContent(DockingNodes, NodeAsArea.ToSharedRef(), NewWindow, OutputCanBeNullptr))
{
    // Create SplitterWidget only if it will be filled with at least 1 DockingNodes
    // Any windows that were "pulled out" of a dock area should be children of the window in which the parent dock area resides.
    if (bIsChildWindow)
    {
        FSlateApplication::Get().AddWindowAsNativeChild(NewWindow, ParentWindow.ToSharedRef())->SetContent(
            SAssignNew(NewDockAreaWidget, SDockingArea, SharedThis(this), NodeAsArea.ToSharedRef()).ParentWindow(NewWindow)
        );
    }
    else
    {
        FSlateApplication::Get().AddWindow(NewWindow)->SetContent(
            SAssignNew(NewDockAreaWidget, SDockingArea, SharedThis(this), NodeAsArea.ToSharedRef()).ParentWindow(NewWindow)
        );
    }
    // Restore content
    if (!bCanOutputBeNullptr)
    {
        RestoreSplitterContent(NodeAsArea.ToSharedRef(), NewDockAreaWidget.ToSharedRef(), NewWindow);
    }
    else
    {
        RestoreSplitterContent(DockingNodes, NewDockAreaWidget.ToSharedRef());
```

知乎 @hareonnna

RestoreSplitterContent の中では **SArea** を走査し、その子要素である **FLayoutNode** を取得します。
ここでの **FLayoutNode** は実際には **FStack** に相当します。これは **FArea** がどのように初期化されているかを参照すると理解できます。

```
TSharedRef<FArea> NewAreaForTab = FTabManager::NewArea(FTabManager::GetDefaultTabWindowSize(TabId))
->Split
(
    FTabManager::NewStack()
    ->AddTab( TabId, ETabState::OpenedTab )
);
```

関係図

ここで再び **RestoreArea_Helper** 関数が再帰的に呼び出され、**FStack** としてその分岐に入ります。

```
if (NodeAsStack.IsValid())
{
    TSharedPtr<SDockTab> WidgetToActivate;

    TSharedPtr<SDockingTabStack> NewStackWidget;
    // Should we init NewStackWidget before the for loop? It depends on OutputCanBeNullptr
    bool bIsNewStackWidgetInit = false;
    // 1. If EOutputCanBeNullptr::Never, function cannot return nullptr
    if (OutputCanBeNullptr == EOutputCanBeNullptr::Never)
    {
        bIsNewStackWidgetInit = true;
    }
    // 2. If EOutputCanBeNullptr::IfNoTabValid, we must init the SWidget as soon as any tab is valid for spawning
    else if (OutputCanBeNullptr == EOutputCanBeNullptr::IfNoTabValid)
    {
        // Note: IsValidTabForSpawning does not check whether SpawnTab() will return nullptr
        for (const FTab& SomeTab : NodeAsStack->Tabs)
        {
            if (IsValidTabForSpawning(SomeTab))
            {
                bIsNewStackWidgetInit = true;
                break;
            }
        }
    }
    // 3. If EOutputCanBeNullptr::IfNoOpenTabValid, we must init the SWidget as soon as any open tab is valid for spawning. For efficiency, done in th
    // 4. Else, case not handled --> error
```

知乎 @hareonnna

ここでは多くの条件分岐が行われていますが、いずれもコメントが付いているため詳細な説明は省略します。重要なのは以下の部分です。

```
}
// Open Tabs
for (const FTab& SomeTab : NodeAsStack->Tabs)
{
    if (SomeTab.TabState == ETabState::OpenedTab && IsValidTabForSpawning(SomeTab))
    {
        const TSharedPtr<SDockTab> NewTabWidget = SpawnTab(SomeTab.TabId, ParentWindow, bCanOutputBeNullptr);

        if (NewTabWidget.IsValid())
        {
            if (SomeTab.TabId == NodeAsStack->ForegroundTabId)
            {
                WidgetToActivate = NewTabWidget;
            }

            // First time initialization: Only if at least a valid NewTabWidget
            if (!NewStackWidget)
            {
                NewStackWidget = SNew(SDockingTabStack, NodeAsStack.ToSharedRef());
                NewStackWidget->SetSizeCoefficient(LayoutNode->GetSizeCoefficient());
            }
            NewStackWidget->AddTabWidget(NewTabWidget.ToSharedRef());
        }
    }
}
```

知乎 @hareonnna

ここで走査しているのは **Stack** 内の **Tabs 配列** です。先ほどの関係図と対応させて見ると、**SpawnTab** 関数では次のような処理が行われます。
まず、**RegisterTabSpawner** によって登録された名前に対応する **FTabSpawnerEntry** が存在するかを検索します。見つかった場合、次に **CanSpawnTab** デリゲートがバインドされているかを確認し、バインドされていればそれを実行します。
続いて **OnSpawnTab** がバインドされているかを確認し、バインドされていれば実行し、**TSharedPtr<SDockTab>** を返します。
その後、この **SDockTab** に対して各種設定を行い、**FTabSpawnerEntry** 内の **SpawnedTabPtr** がこの **SDockTab** を指すように設定されます。
これにより、次回タブを開く際に、前回開かれた Tab かどうかを判定できるようになります。つまり、冒頭の **Invoke** 呼び出し時に行われる判定につながります。

```
bool bSpawningAllowedBySpawner = true;
// Do we know how to spawn such a tab?
TSharedPtr<FTabSpawnerEntry> Spawner = FindTabSpawnerFor(TabId.TabType);
if ( Spawner.IsValid() )
{
    if (Spawner->CanSpawnTab.IsBound())
    {
        bSpawningAllowedBySpawner = Spawner->CanSpawnTab.Execute(FSpawnTabArgs(ParentWindow, TabId));
    }

    if (bSpawningAllowedBySpawner && (!Spawner->SpawnedTabPtr.IsValid() || Spawner->OnFindTabToReuse.IsBound()))
    {
        NewTabWidget = Spawner->OnSpawnTab.Execute(FSpawnTabArgs(ParentWindow, TabId));
        NewTabWidget->SetLayoutIdentifier(TabId);
        NewTabWidget->ProvideDefaultLabel(Spawner->GetDisplayName().IsEmpty() ? FText::FromName(Spawner->TabType) : Spawner->GetDisplayName());
        NewTabWidget->ProvideDefaultIcon(Spawner->GetIcon().GetIcon());

        // The spawner tracks that last tab it spawned
        Spawner->SpawnedTabPtr = NewTabWidget;
    }
    else
    {
        // If we got here, somehow there is two entries spawning the same tab.  This is now allowed so just ignore it.
        bSpawningAllowedBySpawner = false;
    }
```

知乎 @hareonnna

最後に **SDockingTabStack** に値が設定されます。その後、**SWidget** は **OnPaint()** または **Paint()** によって描画される際、**FArea** を起点として、子要素（children）へと階層的に処理が進んでいきます。

InsertNewDocumentTab

エディタ生成時に呼び出された **InsertNewDocumentTab** 関数に戻ります。ここでは、直接 **OpenUnmanagedTab** 関数の中へ入っていきます。

```
void FTabManager::OpenUnmanagedTab(FName PlaceholderId, const FSearchPreference& SearchPreference, const TSharedRef<SDockTab>& UnmanagedTab)
{
    TSharedPtr<SDockTab> LiveTab = SearchPreference.Search(*this, PlaceholderId, UnmanagedTab);

    if (LiveTab.IsValid())
    {
        LiveTab->GetParent()->GetParentDockTabStack()->OpenTab( UnmanagedTab );
    }
    else
    {
        TSharedPtr<SDockingTabStack> StackToSpawnIn = FindPotentiallyClosedTab( PlaceholderId );
        if (StackToSpawnIn.IsValid())
        {
            StackToSpawnIn->OpenTab(UnmanagedTab);
        }
        else
        {
            UE_LOG(LogTabManager, Warning, TEXT("Unable to insert tab '%s'."), *(PlaceholderId.ToString()));
            LiveTab = InvokeTab_Internal( FTabId( PlaceholderId ) );
            check(LiveTab.IsValid());
            if (LiveTab)
            {
                LiveTab->GetParent()->GetParentDockTabStack()->OpenTab( UnmanagedTab );
            }
        }
```

知乎 @hareonnna

引数として渡される **Search** は **FLiveTabSearch** であり、既に開かれている **Tab** が存在するかどうかを検索するためのものです。

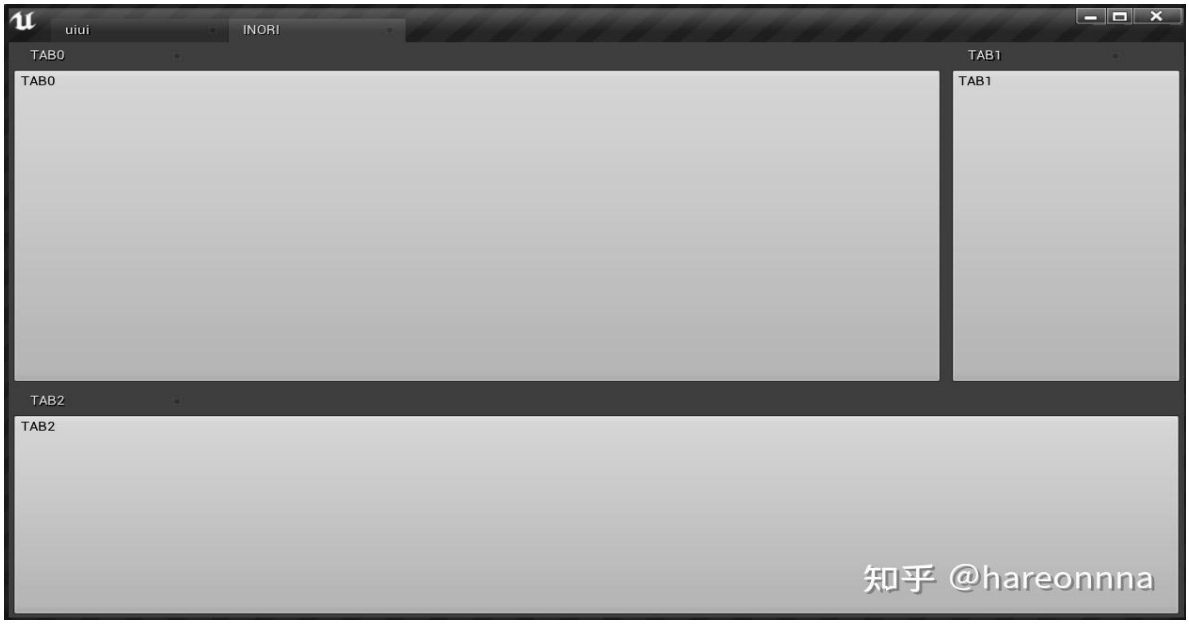
既に開かれている Tab が見つかった場合は、新しいウィンドウを生成するのではなく、その Tab に直接挿入されます。

```
TSharedPtr<SDockTab> FTabManager::FLiveTabSearch::Search(const FTabManager& Manager, FName PlaceholderId, const TSharedRef<SDockTab>& UnmanagedTab) const
{
    if ( SearchForTabId != NAME_None )
    {
        return Manager.FindExistingLiveTab(FTabId(SearchForTabId));
    }
    else
    {
        return Manager.FindExistingLiveTab(FTabId(PlaceholderId));
    }
}
```

知乎 @hareonnna

これは **LayoutIdentifier** を用いて比較されます。

InvokeTab 内で呼び出される **SpawnTab** と、**InsertNewDocumentTab** の冒頭では、それぞれの **SDockTab** に対して **LayoutIdentifier** が設定されています。



例えば、渡される **PlaceholderId** が「uiui」の場合、その隣に新しい Tab が直接挿入されます。
OpenTab の呼び出し時には、すでに **SDockTab** から対応する **SDockingTabStack** が特定されており、**OpenTab** によって新しい **SDockTab** が **SDockingTabStack** の Tabs 配列に追加されます。
もし渡された **FName** に対応するアクティブな　　が見つからず、さらに **FindPotentiallyClosedTab** によって過去に閉じられた Tab も見つからない場合は、**InvokeTab** が呼び出され、新しいウィンドウが作成されます。

UE Slate 学習ノート 4 - カスタムエディタを開く④

この節では、カスタム内容の表示 について説明します。
UE の UI は基本的にすべて **SWidget** を通してエディタ画面に表示されます。**SWidget** の生成は主に **SNew** または **SAssignNew** マクロを用いて行われます。まずはこの 2 つのマクロを確認します。
FAssetEditorToolkit::InitAssetEditor の中を見ると、次のような処理が確認できます。

```
NewMajorTab->SetContent
(
    SAssignNew( NewStandaloneHost, SStandaloneAssetEditorToolkitHost, NewTabManager, AppIdentifier )
    .OnRequestClose(this, &FAssetEditorToolkit::OnRequestClose)
);
```

最初の引数は、値が代入される変数として使用されます。
2 番目の引数は、その変数のクラスです。後ろの 2 つの引数は、そのクラスの **Construct** に基づいて指定します。
ここでは **SStandaloneAssetEditorToolkitHost::Construct** に対応しているため、**FTabManager** と **FName** を渡します。

```
void Construct( const FArguments& InArgs, const TSharedPtr<FTabManager>& InTabManager, const FName InitAppName );
```

SStandaloneAssetEditorToolkitHost::Construct

なぜ引数を **Construct** に基づいて指定する必要があるかというと、オーバーロードされた <<= 演算子が関係しています。
SNew や **SAssignNew** を使用する際、**<<=** によって **WidgetType::FArguments()** へ値を設定できるようになっており、例えば上図の **.OnRequestClose** のように「.」を使った形で引数を指定できます。
そして **<<=** の内部では、そのクラスの **Construct** 関数が呼び出され、渡された引数の内容が実際にウィジェットへ設定されます。

```
TSharedRef<WidgetType> operator<<=( const typename WidgetType::FArguments& InArgs ) const
{
    //@todo UMG: This should be removed in favor of all widgets calling their superclass construct.
    _Widget->SWidgetConstruct(
        InArgs._ToolTipText,
        InArgs._ToolTip,
        InArgs._Cursor,
        InArgs._IsEnabled,
        InArgs._Visibility,
        InArgs._RenderOpacity,
        InArgs._RenderTransform,
        InArgs._RenderTransformPivot,
        InArgs._Tag,
        InArgs._ForceVolatile,
        InArgs._Clipping,
        InArgs._FlowDirectionPreference,
        InArgs._AccessibleText.IsSet() ? FAccessibleWidgetData(InArgs._AccessibleText) : InArgs._AccessibleParams,
        InArgs.MetaData );

    _RequiredArgs.CallConstruct(_Widget, InArgs);

    return _Widget;
}
```

知乎 @hareonnna

次に、独自の **SWidget** を定義します。継承先としては、**SCompoundWidget**、**SPanel**、**SLeafWidget** の 3 種類があります。ここでは、エンジン内の **SButton** の実装を参考にしながら、自分用に書き換えていきます。

SLATE_BEGIN_ARGS() と **SLATE_END_ARGS()** は必須です。

この 2 つのマクロの間で、**SLATE_ARGUMENT**、**SLATE_ATTRIBUTE**、**SLATE_EVENT** を使って、**SButton** 用の変数やデリゲートを宣言します。

また、今回は **SCompoundWidget** を継承しているため、**Slot** を持つことができます。そのため、子となる **SWidget** を格納するための **SLATE_DEFAULT_SLOT** も宣言する必要があります。

```
SLATE_BEGIN_ARGS( SButton )
    : _Content()
    , _ButtonStyle( &FCoreStyle::Get().GetWidgetStyle< FButtonStyle >( "Button" ) )
    , _TextStyle( &FCoreStyle::Get().GetWidgetStyle< FTextBlockStyle >("NormalText") )
    , _HAlign( HAlign_Fill )
    , _VAlign( VAlign_Fill )
    , _ContentPadding(FMargin(4.0, 2.0))
    , _Text()
    , _ClickMethod( EButtonClickMethod::DownAndUp )
    , _TouchMethod( EButtonTouchMethod::DownAndUp )
    , _PressMethod( EButtonPressMethod::DownAndUp )
    , _DesiredSizeScale( FVector2D(1,1) )
    , _ContentScale( FVector2D(1,1) )
    , _ButtonColorAndOpacity(FLinearColor::White)
    , _ForegroundColor( FCoreStyle::Get().GetSlateColor( "InvertedForeground" ) )
    , _IsFocusable( true )
    {
    }

    /** Slot for this button's content (optional) */
    SLATE_DEFAULT_SLOT( FArguments, Content )

    /** The visual style of the button */
    SLATE_STYLE_ARGUMENT( FButtonStyle, ButtonStyle )
```

知乎 @hareonnna

Construct 関数の中では、このクラスのメンバ変数に値を設定するだけでなく、**Slot** がある場合には、その **Slot** 内の内容を親クラスにも渡します。これにより、**OnPaint()** 関数で取得して描画できるようになります。

```
struct
{
    TSharedRef<SWidget> operator()( const FArguments& InOpArgs ) const
    {
        if ((InOpArgs._Content.Widget == SNullWidget::NullWidget) && (InOpArgs._Text.IsBound() || !InOpArgs._Text.Get().IsEmpty()) )
        {
            return SNew(STextBlock)
                .Visibility(EVisibility::HitTestInvisible)
                .Text( InOpArgs._Text )
                .TextStyle( InOpArgs._TextStyle )
                .TextShapingMethod( InOpArgs._TextShapingMethod )
                .TextFlowDirection( InOpArgs._TextFlowDirection );
        }
        else
        {
            return InOpArgs._Content.Widget;
        }
    }
} DetermineContent;

SBorder::Construct( SBorder::FArguments()
    .ContentScale(InArgs._ContentScale)
    .DesiredSizeScale(InArgs._DesiredSizeScale)
    .BorderBackgroundColor(InArgs._ButtonColorAndOpacity)
    .ForegroundColor(InArgs._ForegroundColor)
    .BorderImage( this, &SButton::GetBorder )
    .HAlign( InArgs._HAlign )
    .VAlign( InArgs._VAlign )
    .Padding( TAttribute<FMargin>(this, &SButton::GetCombinedPadding) )
    .ShowEffectWhenDisabled( TAttribute<bool>(this, &SButton::GetShowDisabledEffect) )
    [
        DetermineContent(InArgs)
    ]
```

知乎 @hareonnna

[] は **SBorder** における演算子のオーバーロードです。

SBorder の **Construct** 関数内でも同様に、**Slot** の内容が親クラスへ渡されます。最終的には **SCompoundWidget** の **ChildSlot** に保存されます。

```
ChildSlot
.HAlign(InArgs._HAlign)
.VAlign(InArgs._VAlign)
.Padding(InArgs._Padding)
[
    InArgs._Content.Widget
];
```

知乎 @hareonnna

SButton の描画が開始されると、まず **FSlateDrawElement::MakeBox** によって Button が描画されます。

その後、**SCompoundWidget::OnPaint** が呼び出され、**SButton** 内の子要素が描画されます。

```
int32 SCompoundWidget::OnPaint( const FPaintArgs& Args, const FGeometry& AllottedGeometry, const FSlateRect& MyCullingRect, FSlateWindowElementList& OutDrawElements, int32 LayerId, const FWidgetStyle& InWidgetStyle,
{

    // A CompoundWidget just draws its children
    FArrangedChildren ArrangedChildren(EVisibility::Visible);
    {
        this->ArrangeChildren(AllottedGeometry, ArrangedChildren);
    }

    // There may be zero elements in this array if our child collapsed/hidden
    if( ArrangedChildren.Num() > 0 )
    {
        check( ArrangedChildren.Num() == 1 );
        FArrangedWidget& TheChild = ArrangedChildren[0];

        FWidgetStyle CompoundedWidgetStyle = FWidgetStyle(InWidgetStyle)
            .BlendColorAndOpacityTint(ColorAndOpacity.Get())
            .SetForegroundColor( GetForegroundColor() );

        int32 Layer = 0;
        {
#if WITH_VERY_VERBOSE_SLATE_STATS
            SCOPE_CYCLE_COUNTER(STAT_ChildPaint);
#endif
            Layer = TheChild.Widget->Paint( Args.WithNewParent(this), TheChild.Geometry, MyCullingRect, OutDrawElements, LayerId + 1, CompoundedWidgetStyle, ShouldBeEnabled( bParentEnabled ) );
        }
```

知乎 @hareonnna

ArrangeChildren の中では、先ほど触れた **SCompoundWidget::ChildSlot** が使用されます。

参考資料としている「**Slate 描画浅析（二）**」によると、**SButton** では **LayerId** の計算や受け渡しまで詳しくは扱われていません。この部分については、その方の記事を詳しく確認すると分かりやすいで

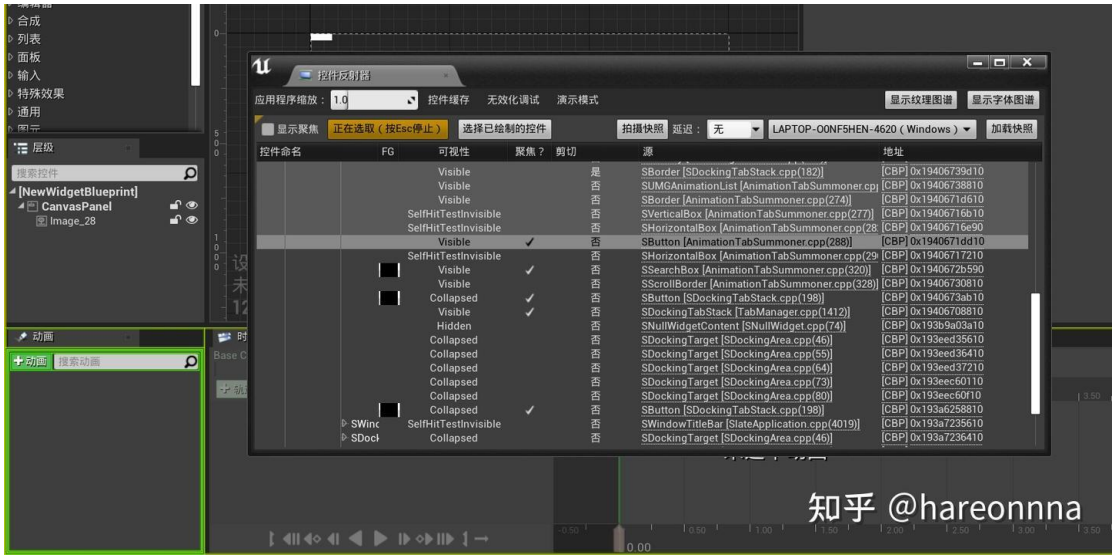
す。

Reference: https://zhuanlan.zhihu.com/p/543267900?utm_id=0

UE Slate 学習ノート 5 - UMG エディタ①（WidgetBlueprintEditor）

Blueprint の UMG では UMG Animation を動的に生成できず、不便に感じる場面が多かったため、UMG エディタがどのように Blueprint + Animation ボタンからアニメーションを生成しているのかを調べることにしました。

まずは Widget Reflector を使って、ソースコードの位置を確認します。



アニメーション追加ボタンのコードは、AnimationTabSummoner.cpp の 288 行目 に書かれていることが分かりました。
また、「调教 UE5：编辑器拓展指南」の Tab 探索 という記事では、FAssetEditorToolkit を継承した FWorkflowCentricApplication クラスについて分析されています。FWorkflowCentricApplication は UMG エディタの親クラスにあたるため、先に理解しておくと分かりやすいです。ここでは簡単にまとめます。
1. FWorkflowCentricApplication の特徴
FWorkflowCentricApplication の特徴は、複数のモード（FApplicationMode）をサポートできることです。
SetCurrentMode 関数によってモードを切り替えることができ、モード切り替えの実体は、エディタ全体のレイアウトや UI 要素を変更することにあります。
例えば UMG エディタ（FWidgetBlueprintEditor）を見ると、2 つのモードを持っていることが分かります。

```
void FWidgetBlueprintEditor::RegisterApplicationModes(const TArray<UBlueprint*>& InBlueprints, bool bShouldOpenInDefaultsMode, bool bNewlyCreated/* = false*/)
{
    //FBlueprintEditor::RegisterApplicationModes(InBlueprints, bShouldOpenInDefaultsMode);

    if ( InBlueprints.Num() == 1 )
    {
        TSharedPtr<FWidgetBlueprintEditor> ThisPtr(SharedThis(this));

        // Create the modes and activate one (which will populate with a real layout)
        TArray< TSharedRef<FApplicationMode> > TempModeList;
        TempModeList.Add(MakeShareable(new FWidgetDesignerApplicationMode(ThisPtr)));
        TempModeList.Add(MakeShareable(new FWidgetGraphApplicationMode(ThisPtr)));

        for ( TSharedRef<FApplicationMode>& AppMode : TempModeList )
        {
            AddApplicationMode(AppMode->GetModeName(), AppMode);
        }

        SetCurrentMode(FWidgetBlueprintApplicationModes::DesignerMode);
```

知乎 @hareonnna

2. 初期状態の FWidgetDesignerApplicationMode モード

初期モードである **FWidgetDesignerApplicationMode** に入ると、コンストラクタ内で独自のページレイアウトが生成されていることが分かります。

そして最も重要なのが、**TabFactories.RegisterFactory** が呼び出されている点です。

```
// Add Tab Spawners
//TabFactories.RegisterFactory(MakeShareable(new FSelectionDetailsSummoner(InWidgetEditor)));
TabFactories.RegisterFactory(MakeShareable(new FDetailsTabSummoner(InWidgetEditor)));
TabFactories.RegisterFactory(MakeShareable(new FDesignerTabSummoner(InWidgetEditor)));
TabFactories.RegisterFactory(MakeShareable(new FHierarchyTabSummoner(InWidgetEditor)));
TabFactories.RegisterFactory(MakeShareable(new FPaletteTabSummoner(InWidgetEditor)));
TabFactories.RegisterFactory(MakeShareable(new FSequencerTabSummoner(InWidgetEditor)));
TabFactories.RegisterFactory(MakeShareable(new FAnimationTabSummoner(InWidgetEditor)));
TabFactories.RegisterFactory(MakeShareable(new FCompilerResultsSummoner(InWidgetEditor)));
```

知乎 @hareonnna

TabFactories は **FWorkflowAllowedTabSet** クラスのインスタンスであり、その内部には **FWorkflowTabFactory** に関する **TMap** が保持されています。

RegisterFactory 関数を使用して、この **TMap** に要素が追加されます。

```
rivate:
    // Map of available factories in this mode
    TMap< FName, TSharedPtr<class FWorkflowTabFactory> > Factories;

    friend class FDocumentTracker;
```

3. FWorkflowTabFactory について

FWorkflowTabFactory は、先ほど Widget Reflector で確認した **AnimationTabSummoner** クラスの親クラスにあたります。

そのため、**FWorkflowTabFactory** に格納されているものは、実際にはページレイアウトを構成する UI 要素となります。

全体の流れは以下の通りです。

SetCurrentMode()

↓

ActivateMode()

↓

RegisterTabFactories()

↓

PushTabFactories ()

知乎 @hareonnna

4. AnimationTabSummoner

PushTabFactories 関数では、**FWorkflowAllowedTabSet** に格納されている Factories を走査し、各 **FWorkflowTabFactory** に対して **RegisterTabSpawner** 関数を呼び出して登録を行います。
ここで重要なのは、**FWorkflowTabFactory::OnSpawnTab → SpawnTab() → CreateTabBody()** という呼び出しの流れです。

つまり、FWorkflowTabFactory で必ずオーバーライドする必要がある CreateTabBody は、ここで使用されていることが分かります。
そして CreateTabBody が返す内容こそが、実際にエディタ上に表示される UI となります。

AnimationTabSummoner の場合、ここでは **SUMGAnimationList** が返されています。

```
TSharedRef<SWidget> FAnimationTabSummoner::CreateTabBody(const FWorkflowTabSpawnInfo& Info) const
{
    TSharedPtr<FWidgetBlueprintEditor> BlueprintEditorPinned = BlueprintEditor.Pin();

    return SNew( SUMGAnimationList, BlueprintEditorPinned );
}
```

知乎 @hareonnna

そして **SUMGAnimationList** が、この Tab の本体となります。

流れを整理したうえで、「+アニメーション」ボタンをクリックした際に何が起こるのかを確認すれば、**UMG Animation** ボタンをどのように動的に生成できるかが分かります。

処理のロジックは **OnNewAnimationClicked** 関数の中にあります。

なお、ここで生成されるのは空の Animation です。Animation の **Track**、**MovieScene**、**Channel** に関する詳細については、「**UE4 動的に UMG Animation を作成する**」という記事を参考にすると理解しやすいです。

Reference: https://zhuanlan.zhihu.com/p/364150799